인공 신경망
Artificial Neural Network

하형진



IDEA Intelligent Design for Engineering Applications Lab

발표 구성 Roadmap



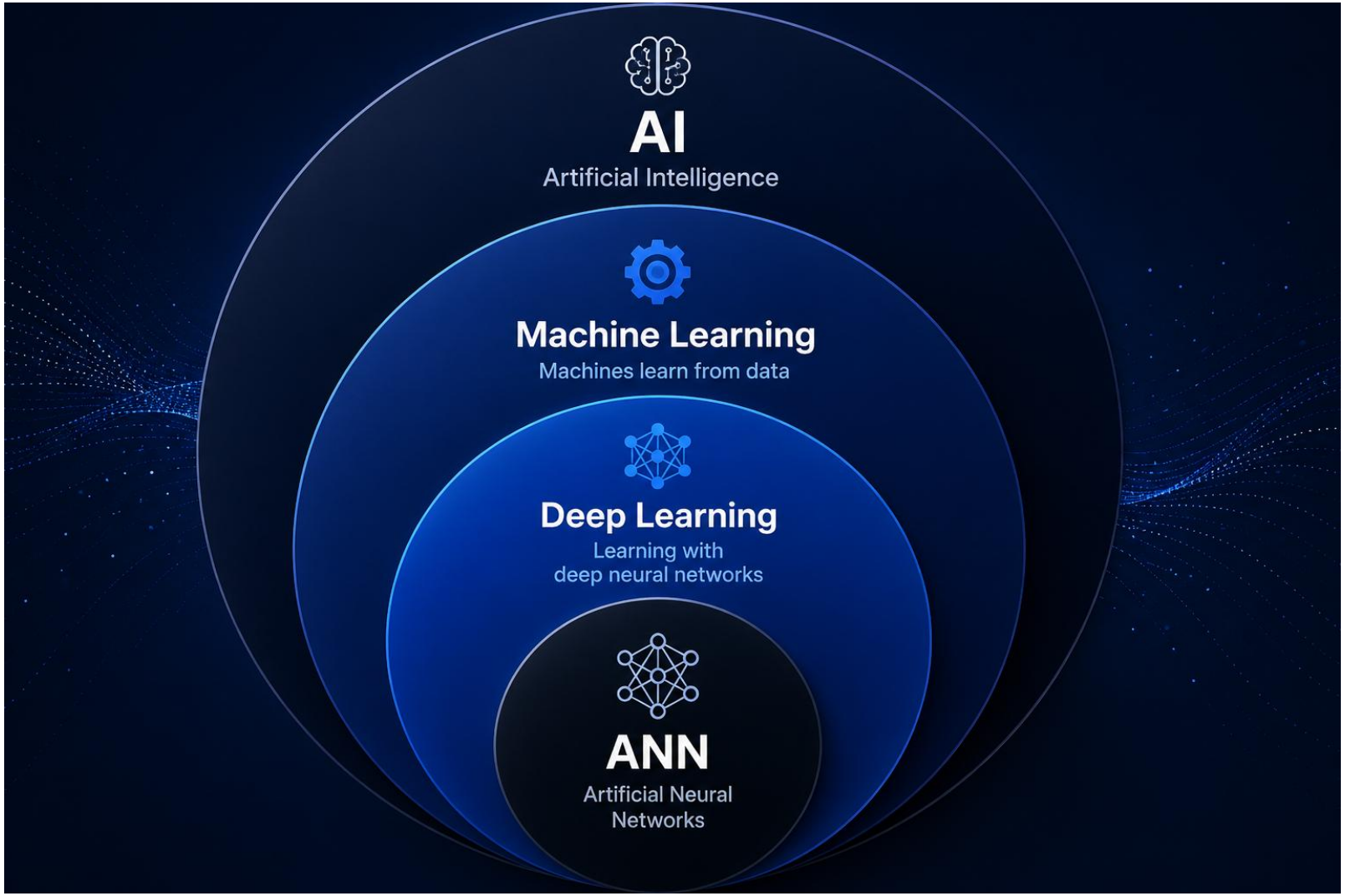
AI·ML·DL ·ANN

전체 코드 흐름

코드
역할별 설명

코드 속
수학적 원리

AI Machine → Machine Learning → Deep Learning → ANN



■ AI

인간의 사고와 판단을 모방하는 기술

■ Machine Learning

데이터를 기반으로 규칙을 스스로 학습

■ Deep Learning

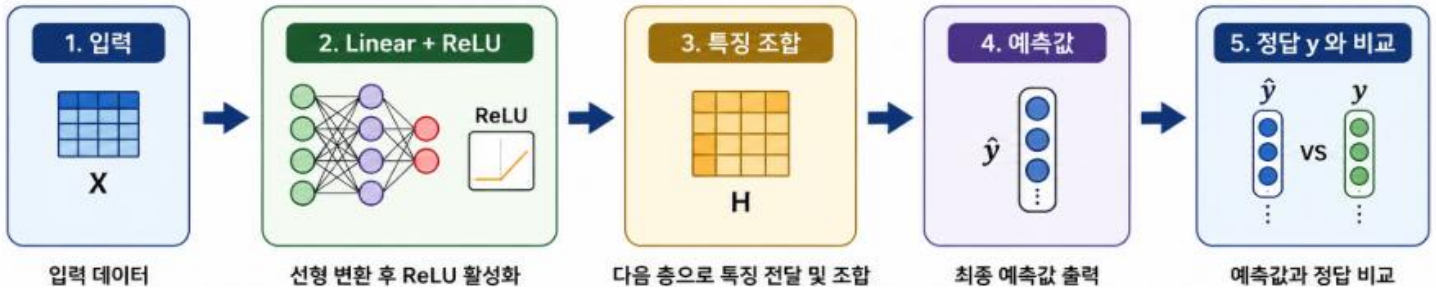
여러 은닉층(Hidden Layer)을 활용한 학습 방법

■ ANN

입력층, 은닉층, 출력층으로 구성된 신경망 모델

Intelligent Design for Engineering Applications Lab

ANN은 무엇을 학습하는가?



■ **인간의 뇌 신경망 구조를 모방하여 컴퓨터가 스스로 패턴을 학습하고 문제를 해결하도록 설계된 머신러닝 알고리즘**

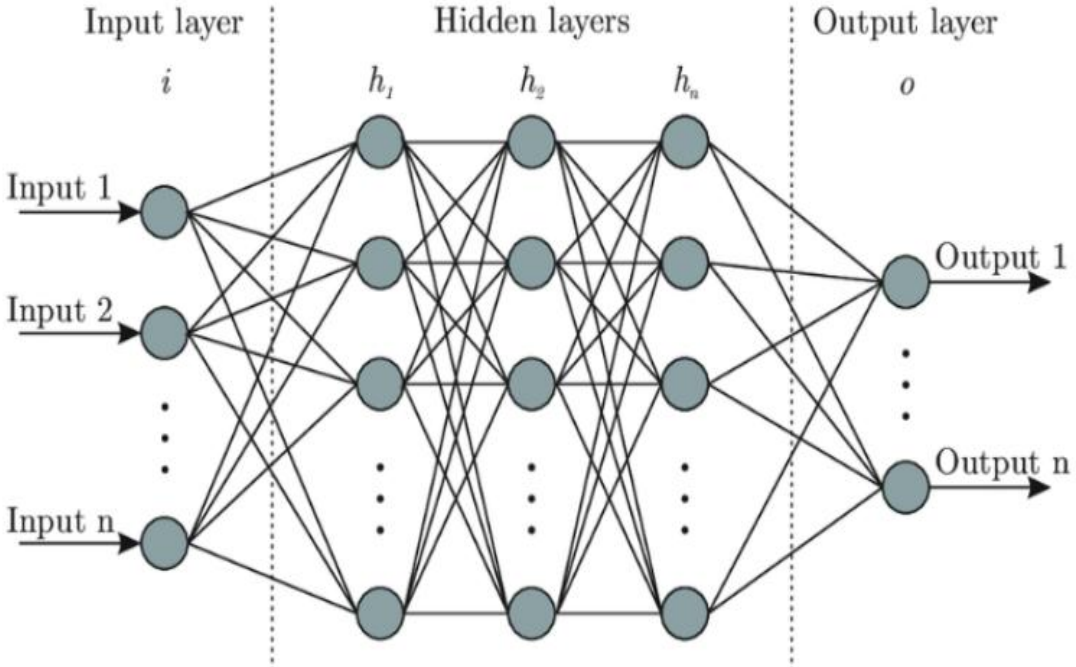
✓ 구성

뉴런(노드): 신경망의 기본 단위, 입력 값을 받아 처리

입력층: 데이터가 네트워크에 들어오는 층,

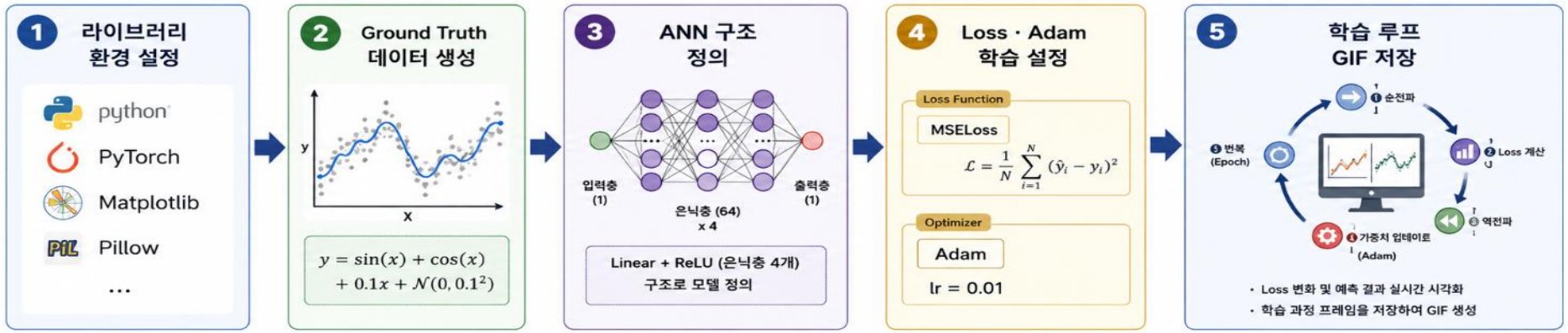
은닉층: 입력층과 출력층 사이에 위치, 데이터의 복잡한 패턴 학습

출력층: 네트워크의 최종 결과를 생성하는 층



Intelligent Design for Engineering Applications Lab

전체 코드 흐름



- torch, numpy, matplotlib 등 필요한 라이브러리 호출
- 그래프 시각화 및 GIF 생성을 위한 환경 구성

- 입력 데이터 x를 생성하고 sin, cos, 선형 추세, 노이즈를 결합하여 실제 공학 데이터와 유사한 복합 진동 데이터 생성
- AI 가 학습해야 하는 정답 데이터를 만드는 과정

- 입력층 1개, 은닉층 4개, 출력층 1개 구성
- nn.Sequential로 Linear + ReLU 모델 생성
- 손실함수(MSELoss) 정의
- Adam Optimizer 설정 (학습률 lr = 0.01)

- EPoch 반복하며 순전파 → Loss → 역전파 →가중치 업데이트 수행
- 10 Epoch마다 그래프를 저장하고 최종적으로 GIF 파일 생성

Intelligent Design for Engineering Applications Lab

1. 라이브러리 호출 및 환경 설정

```
import torch

import torch.nn as nn

import torch.optim as optim

import matplotlib.pyplot as plt

import numpy as np

from IPython.display import clear_output

import time
```

torch	Tensor 연산, 자동미분(Autograd), GPU 연산
torch.nn	Linear, ReLU, Module, Sequential 등 신경망 구조 정의
torch.optim	Adam, SGD 등 최적화 알고리즘 제공
matplotlib	Loss 및 예측 곡선 시각화
numpy	학습 데이터 생성 및 수치 계산
clear_output / time	실시간 그래프 갱신 및 애니메이션 효과 구현

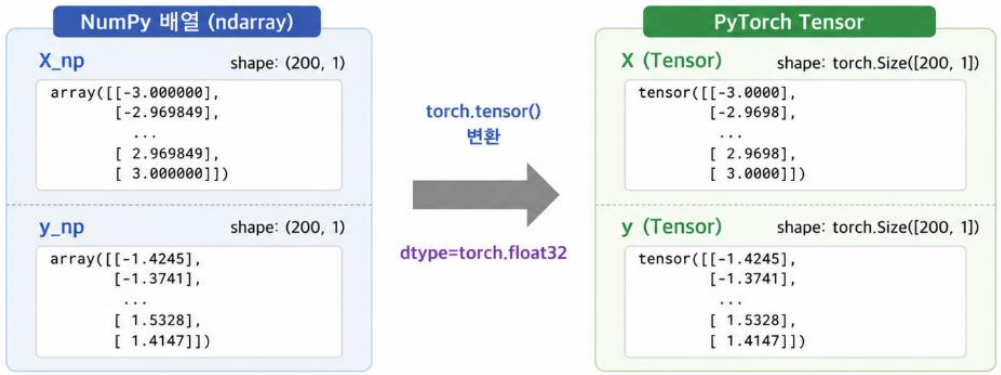
```
np.random.seed(42)
torch.manual_seed(42)
```

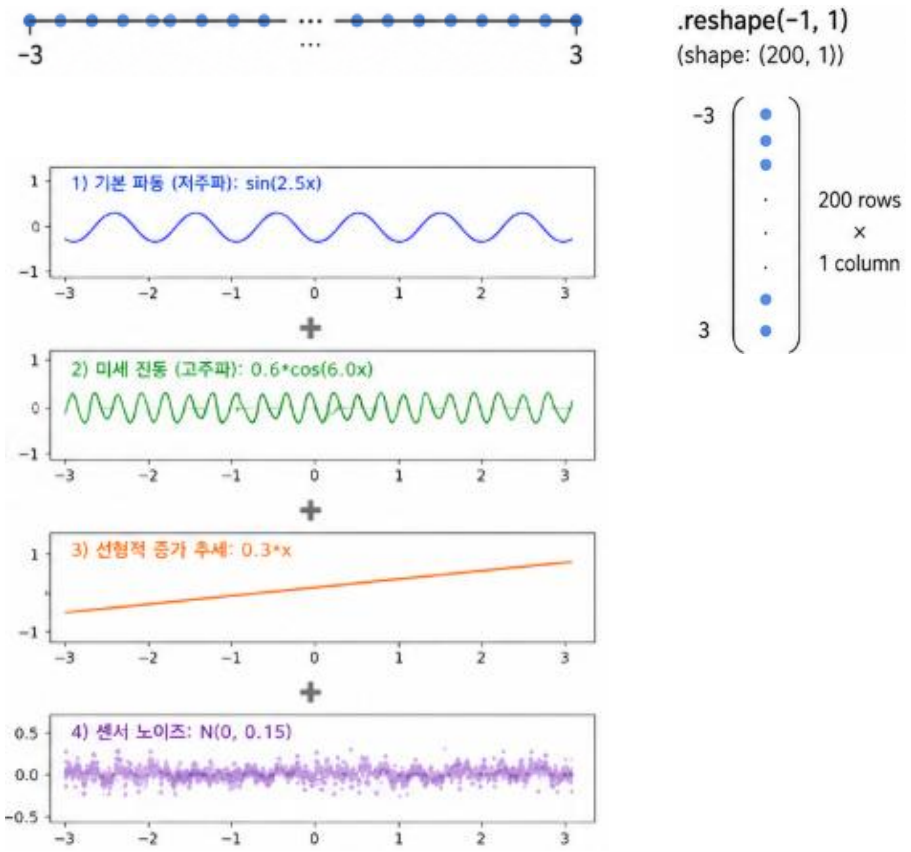
시드는 난수를 없애는 것이 아니라, 난수가 생성되는 순서를
고정하여 매번 동일한 실험 환경을 만드는 역할

Intelligent Design for Engineering Applications Lab

2. Ground Truth 데이터 생성

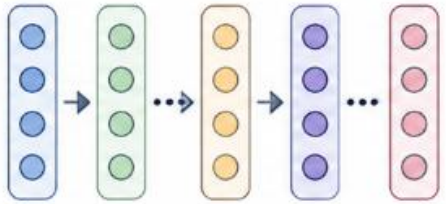
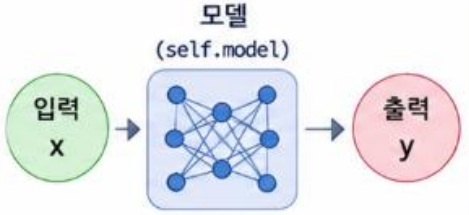
```
X_np = np.linspace(-3, 3, 200).reshape(-1, 1)
y_np = (np.sin(2.5 * X_np)
        + 0.6 * np.cos(6.0 * X_np)
        + 0.3 * X_np
        + np.random.normal(0, 0.15,
size=X_np.shape))
X = torch.tensor(X_np, dtype=torch.float32)
y = torch.tensor(y_np, dtype=torch.float32)
```





Intelligent Design for Engineering Applications Lab

class · model · self · forward

class	model	self	forward
`class SimplePredictiveAI(nn.Module):`	`self.model = nn.Sequential(...)`	`self`	`def forward(self, x):` `  return self.model(x)`
AI 설계도(클래스)를 정의 • nn.Module을 상속받아 모델의 기본 구조를 정의 • 모델의 설계도(틀)을 만드는 역할	**설계도로 만든 실제 AI 객체** • 신경망의 층(레이어)들을 정의하고 연결하여 실제 모델을 구성 • AI가 학습할 구조(네트워크)를 결정	**현재 객체(클래스의 인스턴스) 자신** • 클래스 내부의 변수나 함수를 현재 객체가 사용하도록 함 • '나 자신'을 가리키는 참조	**입력 x가 모델을 통과하는 과정** • 입력 데이터 x를 받아 모델을 통과시켜 출력 생성 • 순전파(Forward Pass) 수행
			
설계도(클래스)	실제 AI 모델(객체)	현재 객체(나 자신)	입력을 모델에 통과 → 출력 생성

Intelligent Design for Engineering Applications Lab

3. ANN 구조 정의

```
class SimplePredictiveAI(nn.Module):

    def __init__(self):

        super(SimplePredictiveAI,
self).__init__()

        self.network = nn.Sequential(

            nn.Linear(1, 64),

            nn.ReLU(),

            nn.Linear(64, 64),

            nn.ReLU(),

            nn.Linear(64, 64),

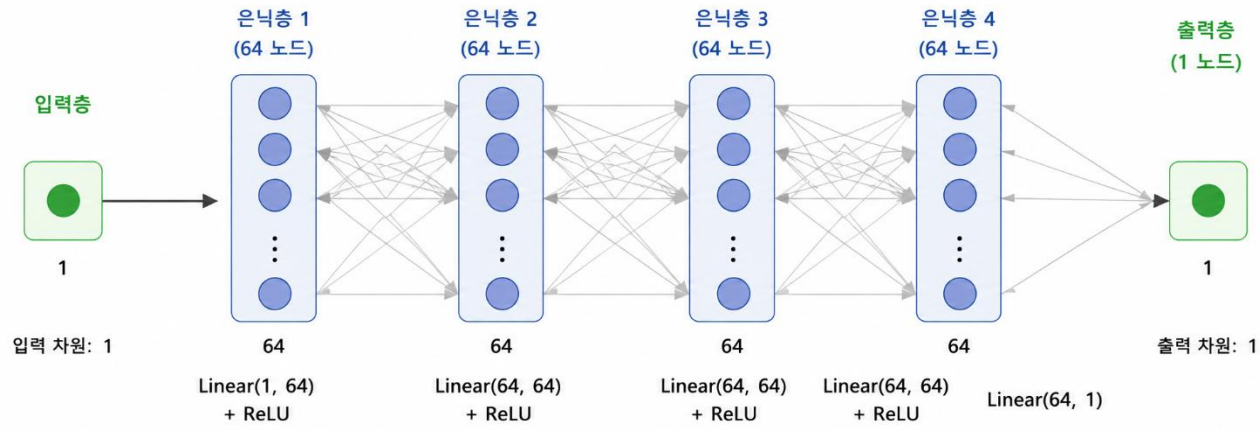
            nn.ReLU(),

            nn.Linear(64, 64),

            nn.ReLU(),

            nn.Linear(64, 1)

        )
```



Intelligent Design for Engineering Applications Lab

Linear · ReLU

Linear(64,64)

64개의 특징을 받아
새로운 64개의 특징으로 조합

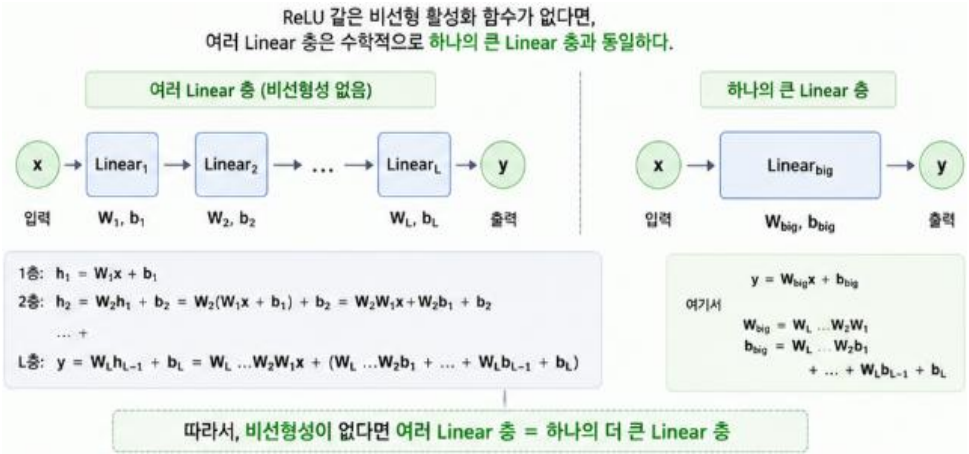


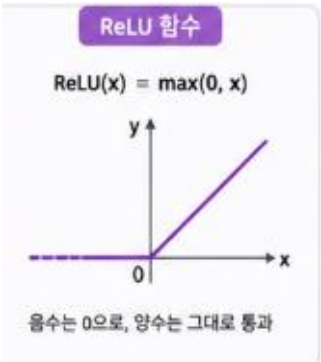
왜 필요한가?

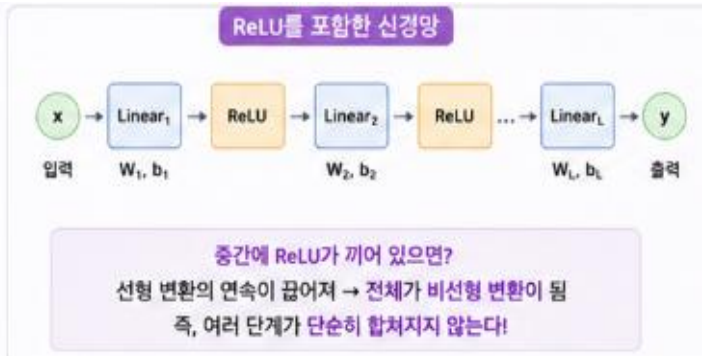
직선 변환 사이에
비선형 분기점을 추가

ReLU()

음수는 0으로 만들고
양수는 그대로 통과

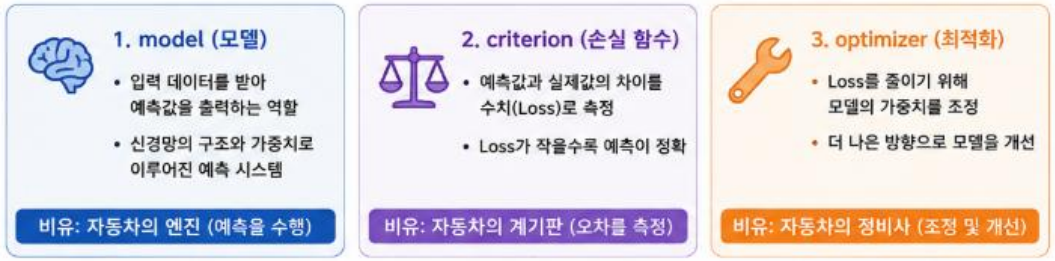






Intelligent Design for Engineering Applications Lab

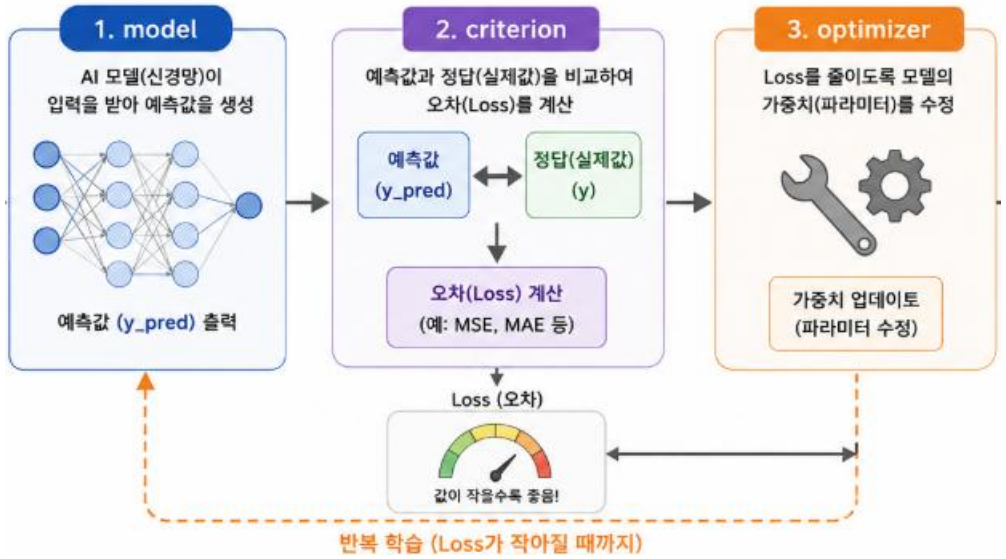
4. 학습 설정(model · 손실함수 · optimizer)




```
model = SimplePredictiveAI()

criterion = nn.MSELoss()

optimizer = optim.Adam(model.parameters(), lr=0.01)
```



Intelligent Design for Engineering Applications Lab

5. 실시간 애니메이션 학습 루프

```
import io

from PIL import Image

from IPython.display import clear_output

import time


epochs = 800

loss_history = []

frames = []


print("🚀 AI 학습 및 GIF 생성 시작 (잠시만 기다려주세요)...")
```

```
for epoch in range(epochs):

    model.train()


    y_pred = model(X)

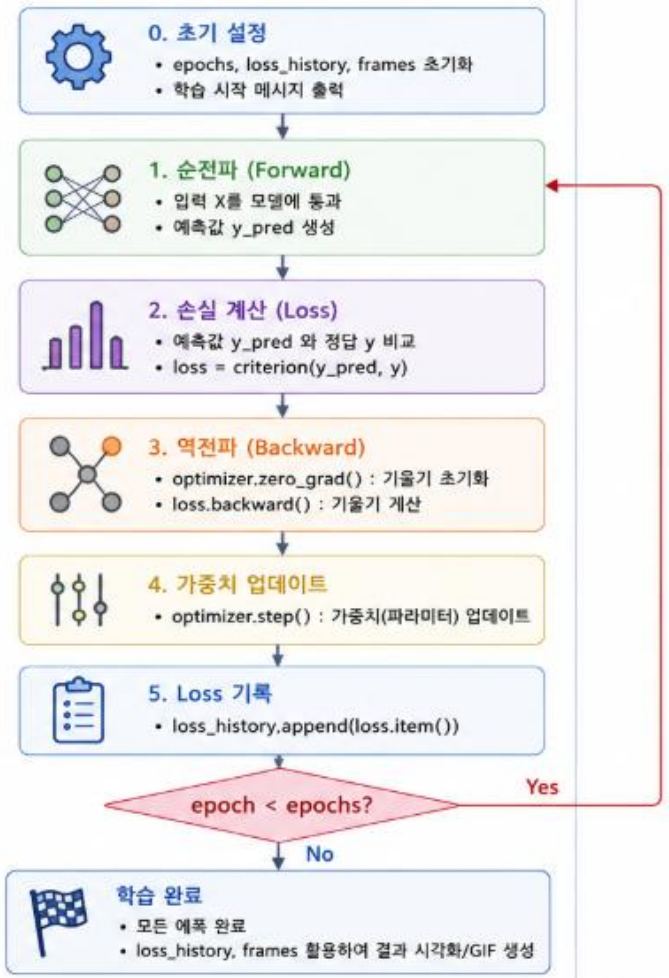
    loss = criterion(y_pred, y)


    optimizer.zero_grad()

    loss.backward()

    optimizer.step()


    loss_history.append(loss.item())
```





Intelligent Design for Engineering Applications Lab

학습 진행 상태 모니터링 및 프레임 저장

```
if (epoch + 1) % 10 == 0 or epoch == 0:
        clear_output(wait=True)
        fig, axes = plt.subplots(1, 2, figsize=(14, 5))

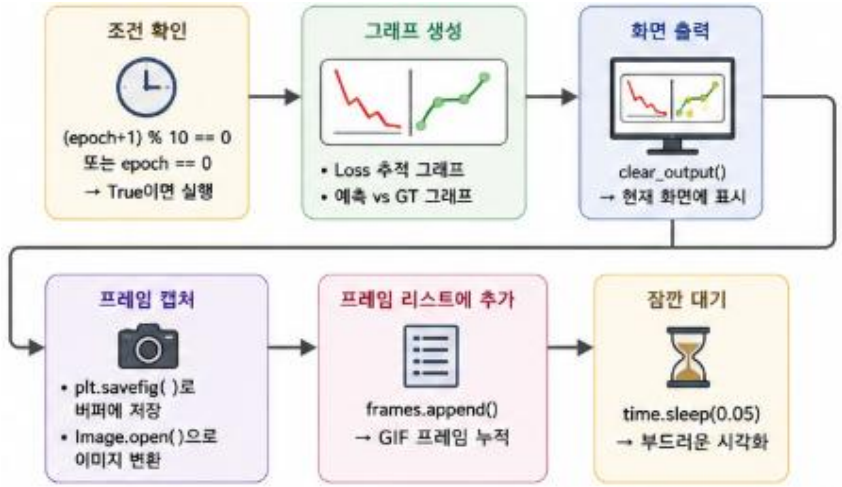
        # [그래프 1] 실시간 Loss 추적기
        axes[0].plot(loss_history, color='#FF5733', linewidth=2, alpha=0.7)
        axes[0].scatter(epoch, loss.item(), color='red', s=100, zorder=5)
        axes[0].set_title(f"1. AI Learning Loss (Epoch: {epoch+1}/{epochs})", fontsize=14, fontweight='bold')
        axes[0].set_xlabel("Epoch", fontsize=12)
        axes[0].set_ylabel("MSE Loss", fontsize=12)
        axes[0].set_xlim(0, epochs)
        axes[0].set_ylim(0, max(loss_history) * 1.1)
        axes[0].grid(True, linestyle='--', alpha=0.6)

        # [그래프 2] 실시간 예측 곡선 변화
        axes[1].scatter(X_np, y_np, color='lightgray', label='Ground Truth (GT)', alpha=0.8)
        axes[1].plot(X_np, y_pred.detach().numpy(), color='green', linewidth=3, label='AI Prediction')
        axes[1].set_title("2. Real-time Prediction vs GT", fontsize=14, fontweight='bold')
        axes[1].set_xlabel("Input Parameter (X)", fontsize=12)
        axes[1].set_ylabel("Engineering Performance", fontsize=12)

        axes[1].set_ylim(min(y_np)-0.5, max(y_np)+0.5)
        axes[1].legend(loc='upper left', fontsize=10)
        axes[1].grid(True, linestyle='--', alpha=0.6)

        plt.tight_layout()
        buf = io.BytesIO()
        plt.savefig(buf, format='png', bbox_inches='tight') # 그래프를 버퍼에 png로 저장
        buf.seek(0)
        frames.append(Image.open(buf)) # PIL 이미지 객체로 변환하여 리스트에 추가

        plt.show() # 주피터 화면에 출력
        time.sleep(0.05)
```



Intelligent Design for Engineering Applications Lab

캡처된 프레임들을 모아 GIF 파일로 저장

```
gif_filename = "AI_Training_Process.gif"

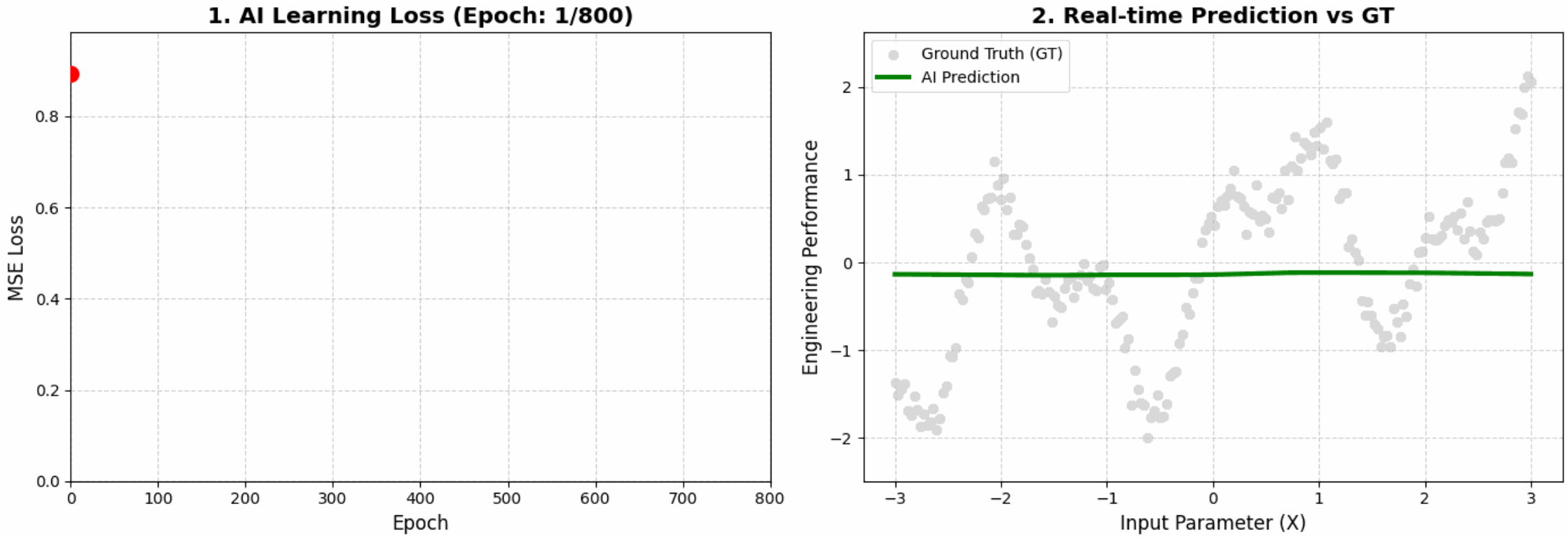

# duration: 한 프레임당 머무는 시간(ms), loop: 0 (무한반복)
frames[0].save(
    gif_filename,
    format='GIF',
    append_images=frames[1:],
    save_all=True,
    duration=80,
    loop=0
)


print(f"\n✨ 학습 완료! 애니메이션이 현재 작업 경로에 '{gif_filename}'
이름으로 저장되었습니다.")
```



Intelligent Design for Engineering Applications Lab

AI_Training_Process.gif



Thank you

